

Bilgisayarlar

Bilgisayar ve Programlamaya Giriş

Veri temsili

Metin

Saklanan program

CPU ve bellek

Soyutlama

Sercan KÜLCÜ · Giresun Üniversitesi

Öğrenme Çıktıları

Sayı temsili

İkili sayıların ve sınırlı bit sayısının sonuçlarını açıklamak.

Metin temsili

ASCII, Unicode ve dizgi kavramlarını ayırt etmek.

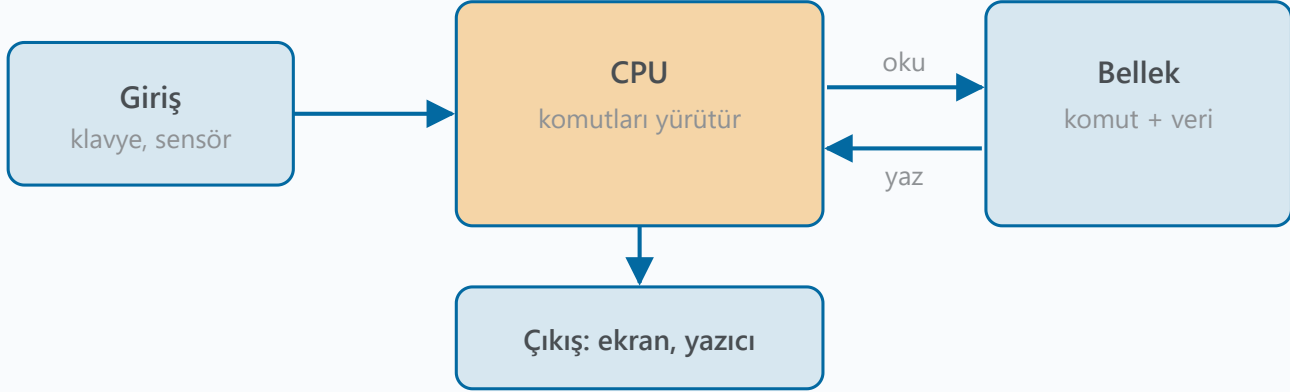
Program yürütme

Saklanan program ve getir-çöz-çalıştır döngüsünü açıklamak.

Mimari

CPU, bellek, yazmaç ve veri yolu ilişkisini yorumlamak.

Bilgisayarın Temel Parçaları



CPU

Komutları yürütür.

Bellek

Veri ve komutları saklar.

Giriş / çıkış

Dış dünya ile veri alışverişi sağlar.

Bilgiden Bitlere

Sayılar, harfler, görüntüler ve program komutları bellekte bit dizileri olarak saklanır. Bit dizisinin anlamını veri tipi ve kodlama belirler.



$$01000001 (65) + 00100000 (32) = 01100001 (97) = \text{"a"}$$

İkili Sayılar



$$1101_2 = 8 + 4 + 1 = 13_{10}$$

n bit, işaretsiz olarak 0 ile $2^n - 1$ arasındaki değerleri gösterebilir.

İkili Sayı: Basamak Değeri ile Hesaplama



Dört bitlik **1011₂** sayısı. Bitler sağdan sola numaralanır: en sağdaki bit 0. bittir.

İkili Sayı: Basamak Değeri ile Hesaplama

Bit	1	0	1	1
	bit 3	bit 2	bit 1	bit 0
Basamak	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$

Her bit konumuna göre bir **2'nin kuvveti** taşır: $2^0 = 1$, $2^1 = 2$, $2^2 = 4$, $2^3 = 8$. Taban 2 olduğu için her basamak sağdaki komşusunun iki katı değer taşır.

İkili Sayı: Basamak Değeri ile Hesaplama

Bit	1	0	1	1
	bit 3	bit 2	bit 1	bit 0
Basamak	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
Çarpım	$1 \times 8 = 8$	$0 \times 4 = 0$	$1 \times 2 = 2$	$1 \times 1 = 1$

Her bit, kendi **basamak değeriyle çarpılır**: bit 1 ise basamak değeri sayıya girer, bit 0 ise hiçbir şey katmaz.

İkili Sayı: Basamak Değeri ile Hesaplama

Bit	1 bit 3	0 bit 2	1 bit 1	1 bit 0
Basamak	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
Çarpım	$1 \times 8 = 8$	$0 \times 4 = 0$	$1 \times 2 = 2$	$1 \times 1 = 1$
$8 + 0 + 2 + 1 = 11$				

Çarpımlar toplanır: $8 + 0 + 2 + 1 = 11$.

İkili Sayı: Basamak Değeri ile Hesaplama

Bit	1	0	1	1
	bit 3	bit 2	bit 1	bit 0
Basamak	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
Çarpım	$1 \times 8 = 8$	$0 \times 4 = 0$	$1 \times 2 = 2$	$1 \times 1 = 1$
$8 + 0 + 2 + 1 = 11$				

$$1011_2 = 11_{10}$$

Sonuç: **$1011_2 = 11_{10}$** . Geri dönüş de aynıdır: $11 = 8 + 2 + 1$ olduğundan 8, 2 ve 1 basamakları 1; 4 basamağı 0 olur.

Taban Kavramı

Sistem	Taban	Rakamlar	Örnek
İkili	2	0–1	1101_2
Sekizli	8	0–7	15_8
Onluk	10	0–9	13_{10}
Onaltılık	16	0–9, A–F	D_{16}

$45_{10} =$ 1 0 1 1 0 1 $_2 \quad (32 + 8 + 4 + 1)$

$45_{10} = 101101_2$. Sekizli ve onaltılık tabanlar, uzun ikili sayıları kısa yazmanın yoludur.

Taban Kavramı

Sistem	Taban	Rakamlar	Örnek
İkili	2	0–1	1101_2
Sekizli	8	0–7	15_8
Onluk	10	0–9	13_{10}
Onaltılık	16	0–9, A–F	D_{16}

$$45_{10} = \boxed{1} \boxed{0} \boxed{1} \boxed{1} \boxed{0} \boxed{1}_2 \quad (32 + 8 + 4 + 1)$$

3'lü grup:

101	101	$\rightarrow 55_8 \quad (5 \cdot 8 + 5 = 45)$
= 5	= 5	

$8 = 2^3$ olduğundan bitler sağdan **üçerli** gruplanır: $101\ 101 \rightarrow 55_8$.

Taban Kavramı

Sistem	Taban	Rakamlar	Örnek
İkili	2	0–1	1101_2
Sekizli	8	0–7	15_8
Onluk	10	0–9	13_{10}
Onaltılık	16	0–9, A–F	D_{16}

$$45_{10} = \boxed{1} \boxed{0} \boxed{1} \boxed{1} \boxed{0} \boxed{1}_2 \quad (32 + 8 + 4 + 1)$$

3'lü grup: $\boxed{101} = 5$ $\boxed{101} = 5 \rightarrow 55_8 \quad (5 \cdot 8 + 5 = 45)$

4'lü grup: $\boxed{0010} = 2 = 2$ $\boxed{1101} = 13 = D \rightarrow 2D_{16} \quad (2 \cdot 16 + 13 = 45)$

$16 = 2^4$ olduğundan **dörderli** gruplanır: $0010 \ 1101 \rightarrow 2$ ve 13 (D) $\rightarrow 2D_{16}$.

Negatif Tam Sayılar

Modern bilgisayarlar negatif tam sayıları çoğunlukla **ikiye tümleyen** gösterimle saklar.

Tek sıfır

Pozitif ve negatif sıfır ayrımı oluşmaz.

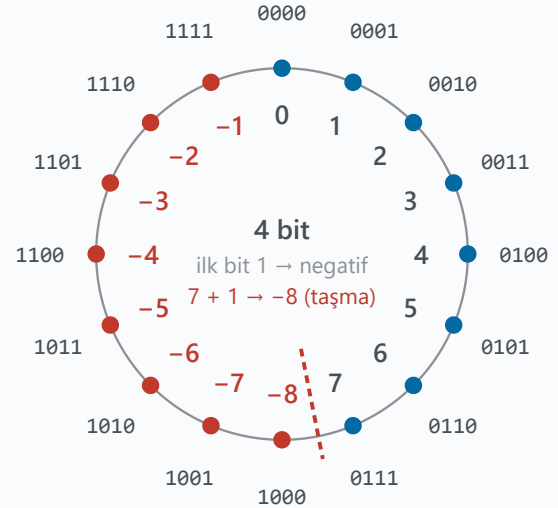
Aritmetik

Toplama devreleri pozitif ve negatif sayılarda aynı kuralları kullanabilir.

n bit aralığı

-2^{n-1} ile $2^{n-1}-1$ arasındadır.

8 bit: -128 ... 127



İkiye Tümleyen: -5 Nasıl Yazılır?

+5



8 bitte **+5 = 0000101**. -5'i bulmak için iki adım yeterli.

Adım 1 / 4

İkiye Tümleyen: -5 Nasıl Yazılır?

+5	0	0	0	0	0	1	0	1
ters çevir	1	1	1	1	1	0	1	0

1. Tüm bitleri **ters çevir** ($0 \leftrightarrow 1$): 11111010. Buna birler tümleyeni denir.

İkiye Tümleyen: -5 Nasıl Yazılır?

+5	0	0	0	0	0	1	0	1
ters çevir	1	1	1	1	1	0	1	0
+1 → -5	1	1	1	1	1	0	1	1

2. Sonuca **1 ekle**: $11111010 + 1 = \mathbf{11111011} = -5$. En soldaki 1, sayının negatif olduğunu gösterir.

İkiye Tömleyen: -5 Nasıl Yazılır?

+5	0	0	0	0	0	1	0	1
-5	1	1	1	1	1	0	1	1
toplam	1	0	0	0	0	0	0	0

taşan elde atılır

8 bitte kalan: 00000000 = 0 ✓

Kontrol: 00000101 + 11111011 = 100000000. 9. bit 8 bite sığmaz ve atılır → 0. Toplama devresi negatifler için ayrıca bir şey yapmaz.

Taşma: 8 Bitte $200 + 100$

200	1	1	0	0	1	0	0	0
+ 100	0	1	1	0	0	1	0	0

8 bit işaretsiz sayı en fazla **255** olabilir. $200 + 100$ toplayalım.

Adım 1 / 3

İşaretli 8 bitte de $127 + 1$ taşar ve -128 olur. Programlarda tip seçimi bu yüzden önemlidir.

Taşma: 8 Bitte 200 + 100

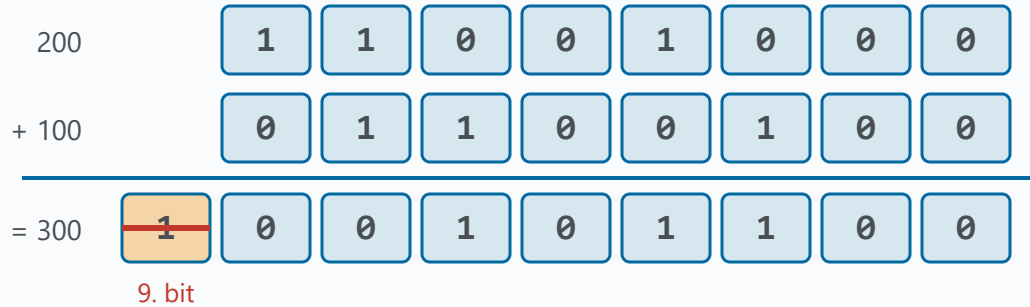
200	1	1	0	0	1	0	0	0
+ 100	0	1	1	0	0	1	0	0
= 300	1	0	0	1	0	1	1	0
	9. bit							

Gerçek toplam **300** = 100101100_2 : sonuç **9 bit** gerektirir.

Adım 2 / 3

İşaretli 8 bitte de $127 + 1$ taşar ve -128 olur. Programlarda tip seçimi bu yüzden önemlidir.

Taşma: 8 Bitte 200 + 100



8 bitte kalan: 00101100 = 44

Yazmaç yalnızca 8 bit tutar; 9. bit (256) kaybolur → sonuç **44**. Buna **taşma (overflow)** denir: $300 - 256 = 44$.

Adım 3 / 3

İşaretili 8 bitte de $127 + 1$ taşar ve -128 olur. Programlarda tip seçimi bu yüzden önemlidir.

Ondalıklı Sayılar

Kayan nokta

Çok büyük ve çok küçük değerleri bilimsel gösterime benzer biçimde saklar.

Sınırlı hassasiyet

Her ondalık değer sonlu bitlerle tam temsil edilemez.

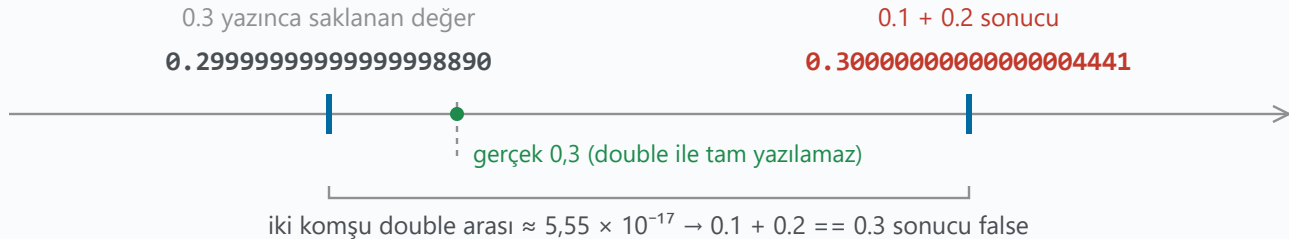
Yuvarlama

Hesaplama sonuçlarında küçük farklar oluşabilir.

Karşılaştırma

Ondalıklı sonuçlar tolerans değeriyle karşılaştırılabilir.

```
0.1 + 0.2 → 0.30000000000000004 // double ile hesaplanınca
```



0,1 Neden Tam Saklanamaz?

Kesri sürekli 2 ile çarp; tam kısım bir sonraki bit

$$0,1_{10} = 0,$$

Ondalık kesir ikiliye nasıl çevrilir? Kesri 2 ile çarp, tam kısmı (0 ya da 1) bit olarak al, kalan kesirle devam et.

0,1 Neden Tam Saklanamaz?

Kesri sürekli 2 ile çarp; tam kısım bir sonraki bit

$$0,1 \times 2 = 0,2$$

bit 0

$$0,1_{10} = 0,0$$

$$0,1 \times 2 = 0,2 \rightarrow \text{tam kısım } \mathbf{0}.$$

0,1 Neden Tam Saklanamaz?

Kesri sürekli 2 ile çarp; tam kısım bir sonraki bit

$$0,1 \times 2 = 0,2$$

bit 0

$$0,2 \times 2 = 0,4$$

bit 0

$$0,1_{10} = 0,00$$

$$0,2 \times 2 = 0,4 \rightarrow \text{tam kısım } \mathbf{0}.$$

0,1 Neden Tam Saklanamaz?

Kesri sürekli 2 ile çarp; tam kısım bir sonraki bit

$$0,1 \times 2 = 0,2$$

bit 0

$$0,2 \times 2 = 0,4$$

bit 0

$$0,4 \times 2 = 0,8$$

bit 0

$$0,1_{10} = 0,000$$

$$0,4 \times 2 = 0,8 \rightarrow \text{tam kısım } \mathbf{0}.$$

0,1 Neden Tam Saklanamaz?

Kesri sürekli 2 ile çarp; tam kısım bir sonraki bit

$$0,1 \times 2 = 0,2$$

bit 0

$$0,2 \times 2 = 0,4$$

bit 0

$$0,4 \times 2 = 0,8$$

bit 0

$$0,8 \times 2 = 1,6$$

bit 1

$$0,1_{10} = 0,0001$$

$0,8 \times 2 = 1,6 \rightarrow$ tam kısım **1**, kesir 1 çıkarılarak devam eder.

0,1 Neden Tam Saklanamaz?

Kesri sürekli 2 ile çarp; tam kısım bir sonraki bit

$$0,1 \times 2 = 0,2$$

bit 0

$$0,2 \times 2 = 0,4$$

bit 0

$$0,4 \times 2 = 0,8$$

bit 0

$$0,8 \times 2 = 1,6$$

bit 1

$$0,6 \times 2 = 1,2$$

bit 1

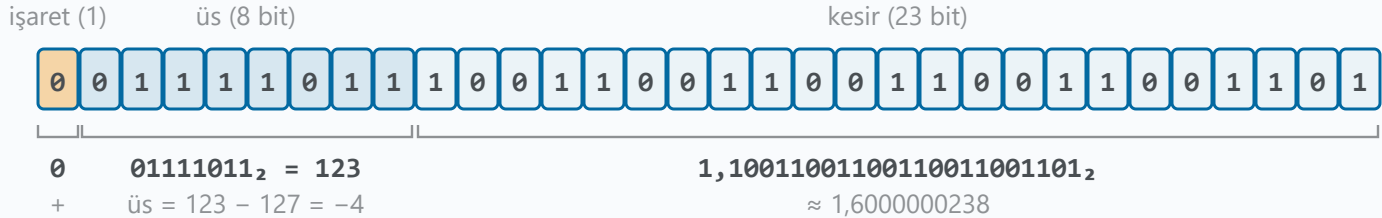
$$0,1_{10} = 0,00011...$$

0,2 tekrar geldi → "0011" sonsuza dek sürer

Sonlu bitle kesilir → küçük yuvarlama hatası

$0,6 \times 2 = 1,2 \rightarrow$ tam kısım **1**, kesir 1 çıkarılarak devam eder. Kesir yine 0,2 oldu: açılım **döngüye girdi**, hiçbir zaman bitmez.

Kayan Noktalı Sayılar (IEEE 754)



$$\text{değer} = (-1)^{\text{işaret}} \times 1,\text{kesir} \times 2^{(\text{üs} - 127)}$$

$$0,1 \text{ (float)} = 0x3DCCCCD \\ \approx 1,6000000238 \times 2^{-4} \approx 0,10000000149$$

- **float** 32 bittir (1 + 8 + 23); **double** 64 bittir (1 + 11 + 52). Normal sayılarda baştaki 1 saklanmaz.
- $0,1 = 1,1001\ 1001\ 1001\dots_2 \times 2^{-4}$: kesir sonsuza dek tekrarlar, 23 bite sığmayınca yuvarlanır.
- Saklanan değer tam olarak 0,100000001490116119384765625'tir: 0,1'e çok yakın ama eşit değil.

ASCII

ASCII, temel Latin harflerini, rakamları, noktalama işaretlerini ve kontrol karakterlerini sayısal kodlarla eşleştirir.

ASCII **7 bitlik** bir koddur: 128 karakter (0–127) tanımlar; bir bayt içinde saklandığında en yüksek bit 0'dır.

A

65

01000001

a

97

01100001

0

48

00110000

Satır sonu (LF)

Kontrol karakteri:

10

00001010

"A" = 65**"a"** = 97

32'ler basamağı

Büyük ve küçük harf
yalnızca bir bitte ayrılır:

$$97 - 65 = 32$$

Unicode

Unicode, farklı yazı sistemlerindeki karakterlere benzersiz kod noktaları atar.

Kod noktası

Karakterin evrensel numarasıdır.

A → U+0041

UTF-8

Unicode karakterlerini değişken uzunlukta bayt dizileriyle kodlar.

Geniş kapsam

Türkçe karakterler, farklı alfabeler, matematik simgeleri ve emoji.

Dosya kodlaması

Aynı kod noktası farklı Unicode dönüşüm biçimleriyle saklanabilir.

karakter	kod noktası	UTF-8 baytları (onaltılık)				uzunluk
A	U+0041	41				1 bayt
ğ	U+011F	C4	9F			2 bayt
€	U+20AC	E2	82	AC		3 bayt
😄	U+1F600	F0	9F	98	80	4 bayt

UTF-8 Kodlama: “ğ”

“ğ” → kod noktası U+011F = 287

Unicode, “ğ” harfine **U+011F** (onluk 287) kod noktasını atar. UTF-8 bu sayıyı baytlara nasıl yazar?

UTF-8 Kodlama: "ğ"

"ğ" → kod noktası U+011F = 287

11 bit:



287 ikili yazılır: 00100011111. 7 bite sığmaz (ASCII değil) ama 11 bite sığar → **2 bayt** gerekir.

UTF-8 Kodlama: "ğ"

"ğ" → kod noktası U+011F = 287

11 bit:



5 bit

6 bit

2 baytlık UTF-8 şablonu **110xxxxx 10xxxxxx**: 11 bit, 5 + 6 olarak ikiye bölünür.

UTF-8 Kodlama: "ğ"

"ğ" → kod noktası U+011F = 287

11 bit:



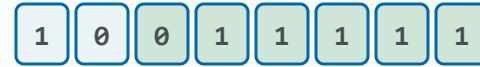
5 bit

6 bit

şablon:



= C4₁₆



= 9F₁₆

Bitler şablona yerleşir: **11000100 10011111** = **C4 9F**. Baştaki 110 ve 10, baytın görevini söyler.

UTF-8 Kodlama: "ğ"

"ğ" → kod noktası U+011F = 287

11 bit:



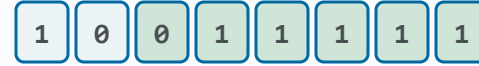
5 bit

6 bit

şablon:



= C4₁₆



= 9F₁₆

Başka örnekler:

A
41 (1 B)

ğ
C4 9F (2 B)

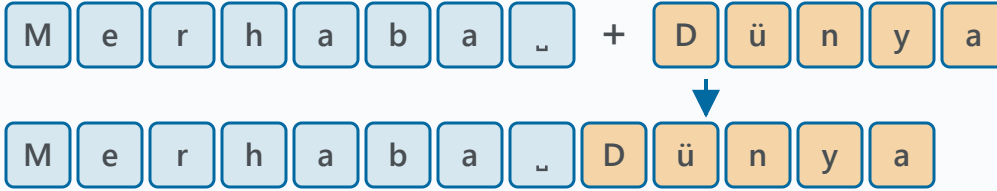
€
E2 82 AC (3 B)

😊
F0 9F 98 80 (4 B)

ASCII karakterleri 1 bayttır; Türkçe harfler 2, € 3, emoji 4 bayt tutar. UTF-8 bu yüzden **değişken uzunluktadır**.

Dizgiler

Dizgi (string), sıralı karakterlerden oluşan veri tipidir.



uzunluk: 13 karakter · UTF-8'de 14 bayt ("ü" 2 bayt)

Uzunluk

Karakter veya kod birimi sayısı.

Birleştirme

İki dizgiyi uç uca ekleme.

Arama

Bir karakteri veya alt dizgiyi bulma.

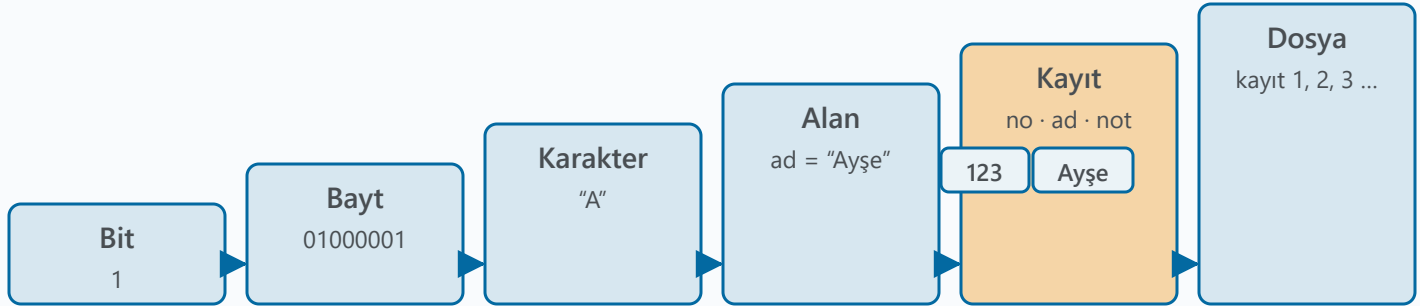
Karşılaştırma

İçerik ve sıralama ilişkisini denetleme.

Ekranada görünen karakter sayısı ile bellekteki bayt (kod birimi) sayısı her zaman aynı değildir.

Yapılandırılmış Bilgi

Bitler daha büyük yapılar içinde düzenlenerek anlam kazanır.



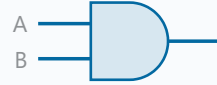
- Bir öğrenci kaydı numara, ad ve not alanlarından oluşabilir.
- Dosya biçimi, alanların nasıl sıralandığını ve yorumlandığını tanımlar.

Mantık Kapıları

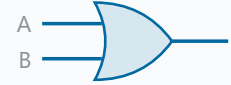
A	B	AND	OR	XOR	NOT A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

- **AND:** iki girdi de 1 ise 1.
- **OR:** en az bir girdi 1 ise 1.
- **XOR:** girdiler farklıysa 1.
- **NOT:** girdiyi tersine çevirir.

Kapılar birkaç transistörden kurulur; işlemcideki tüm işlemler bu temel kapıların birleşiminden oluşur.



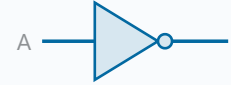
AND (VE)



OR (VEYA)

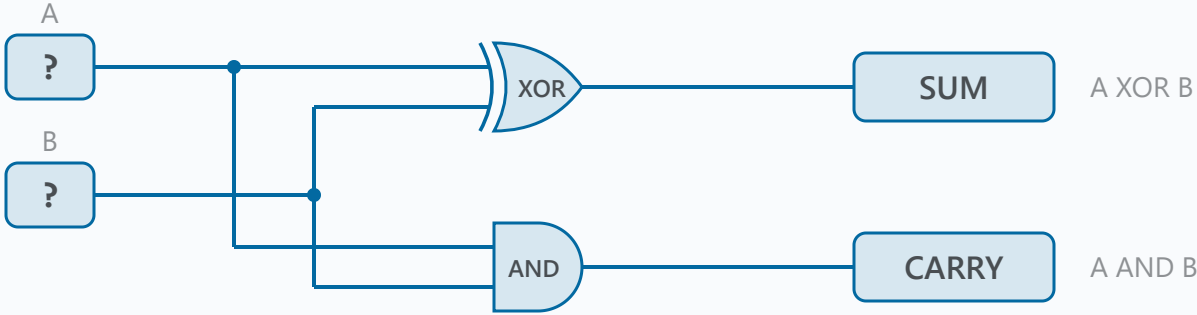


XOR (ÖZEL VEYA)



NOT (DEĞİL)

Yarım Toplayıcı: Kapılardan Toplamaya



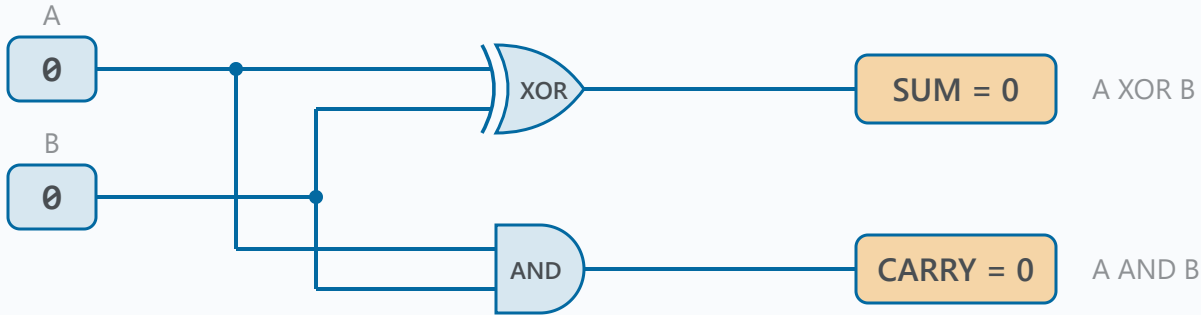
Sonuç iki bitle yazılır: CARRY SUM

Yarım toplayıcı iki tek bitlik sayıyı (A, B) toplar. **SUM = A XOR B** sonuç basamağını, **CARRY = A AND B** eldeyi verir. Yalnızca iki kapı yeterlidir.

Adım 1 / 4

Yarım toplayıcı gelen eldeyi almaz. Çok bitli toplama için üç girişli **tam toplayıcı** gerekir (iki yarım toplayıcı ve bir OR kapısı).

Yarım Toplayıcı: Kapılardan Toplamaya



$$0 + 0 = 00_2 = 0$$

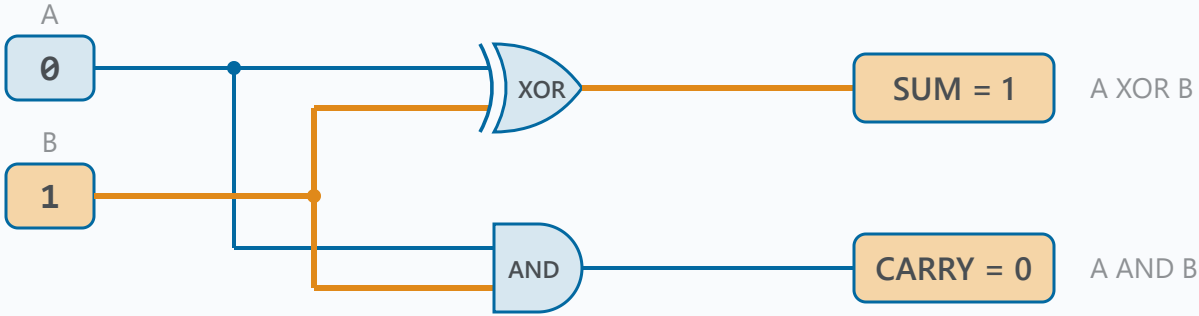
(CARRY, SUM) sırasıyla yazılır

$$0 + 0: \text{XOR}(0, 0) = 0, \text{AND}(0, 0) = 0 \rightarrow 00_2 = 0.$$

Adım 2 / 4

Yarım toplayıcı gelen eldeyi almaz. Çok bitli toplama için üç girişli **tam toplayıcı** gerekir (iki yarım toplayıcı ve bir OR kapısı).

Yarım Toplayıcı: Kapılardan Toplamaya



$$0 + 1 = 01_2 = 1$$

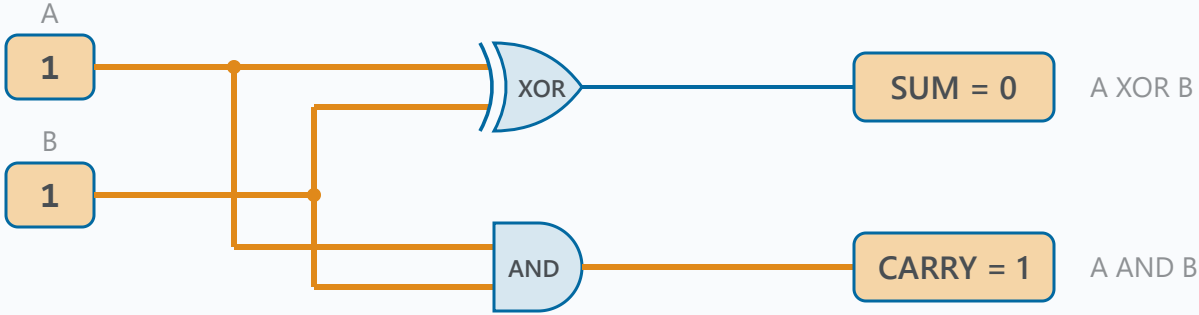
(CARRY, SUM) sırasıyla yazılır

$0 + 1$: $\text{XOR}(0, 1) = 1$, $\text{AND}(0, 1) = 0 \rightarrow 01_2 = 1$. Elde oluşmadı.

Adım 3 / 4

Yarım toplayıcı gelen eldeyi almaz. Çok bitli toplama için üç girişli **tam toplayıcı** gerekir (iki yarım toplayıcı ve bir OR kapısı).

Yarım Toplayıcı: Kapılardan Toplamaya



$$1 + 1 = 10_2 = 2$$

(CARRY, SUM) sırasıyla yazılır

1 + 1: XOR(1, 1) = 0 (basamak 0), AND(1, 1) = 1 (elde 1) → **10₂ = 2**. Onluk sistemdeki 9 + 1 = 10 gibi, elde bir üst basamağa gider.

Adım 4 / 4

Yarım toplayıcı gelen eldeyi almaz. Çok bitli toplama için üç girişli **tam toplayıcı** gerekir (iki yarım toplayıcı ve bir OR kapısı).

Depolanan Program Kavramı

Bellek		
1000	LOAD 1004	program
1001	ADD 1005	
1002	STORE 1006	
1003	HALT	
1004	12	veri
1005	8	
1006	0	

aynı bellek, aynı adres sistemi

Program komutları ve işlenecek veriler **aynı bellekte** sayısal kodlar olarak saklanır. İşlemci komutları sırayla okuyup yürütür.

Esneklik

Aynı donanım farklı programlarla farklı görevler yapar.

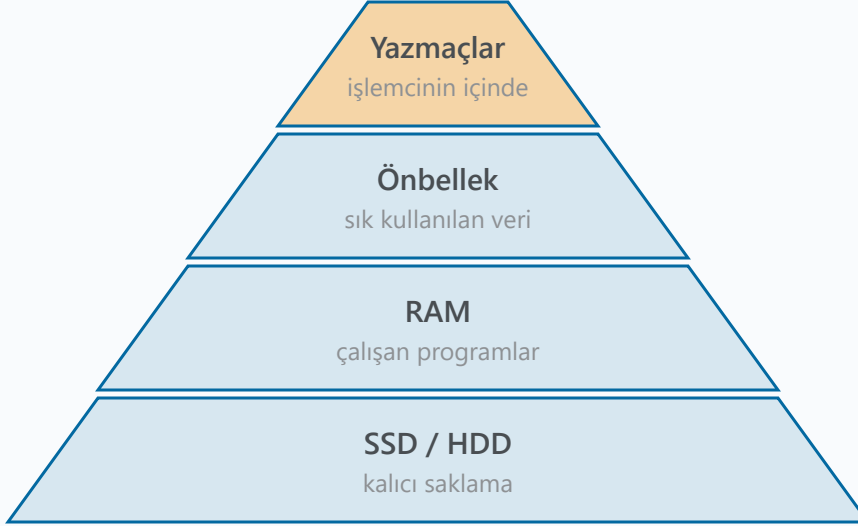
Güncelleme

Davranış donanımı değiştirmeden yazılımla değişebilir.

Genel amaçlılık

Tek sistem hesaplama, iletişim ve medya görevlerini yürütebilir.

Bellek Hiyerarşisi



boyut · erişim süresi

birkaç yüz bayt
< 1 ns

KB – MB
~1–10 ns

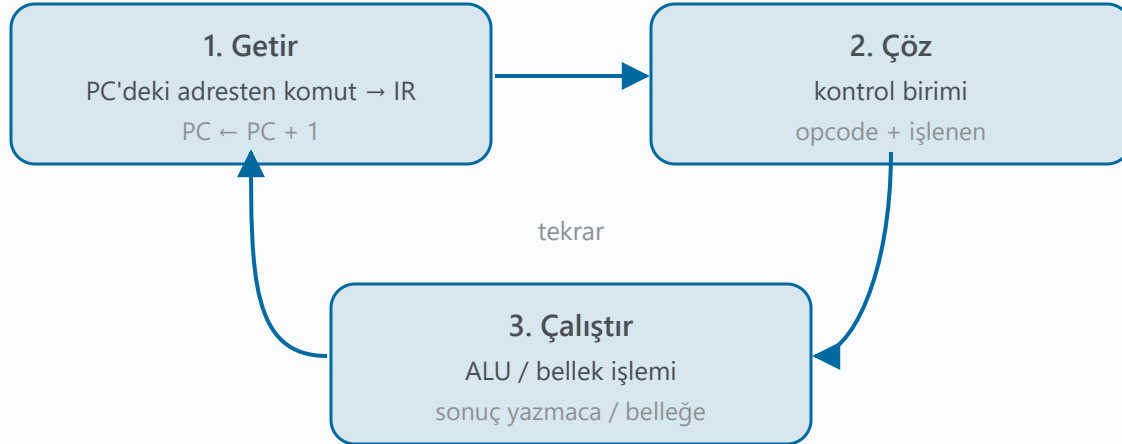
GB
~100 ns

TB
SSD ~0,1 ms · HDD ~10 ms

daha hızlı · daha pahalı

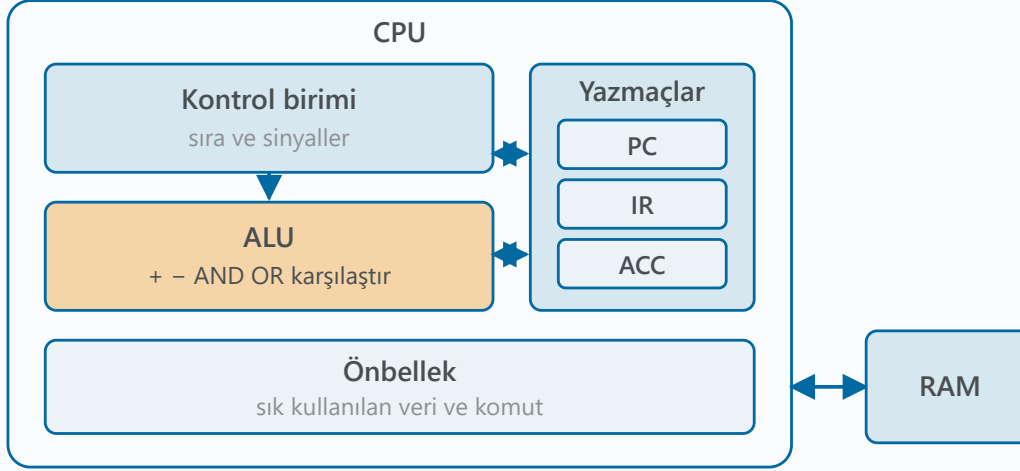
Üst katmanlar daha hızlı ve küçük; alt katmanlar daha yavaş ve büyüktür. Süreler yaklaşık büyüklük mertebesidir.

Getir-Çöz-Çalıştır Döngüsü



- **Program sayacı (PC)** sıradaki komutun adresini, **komut yazmacı (IR)** getirilen komutu tutar.
- Kontrol birimi işlem kodunu (opcode) yorumlar.
- ALU veya başka bir birim işlemi gerçekleştirir.

CPU'nun Temel Bölümleri



Kontrol birimi

Komutların sırasını ve veri hareketini yönetir.

ALU

Aritmetik ve mantıksal işlemleri yapar.

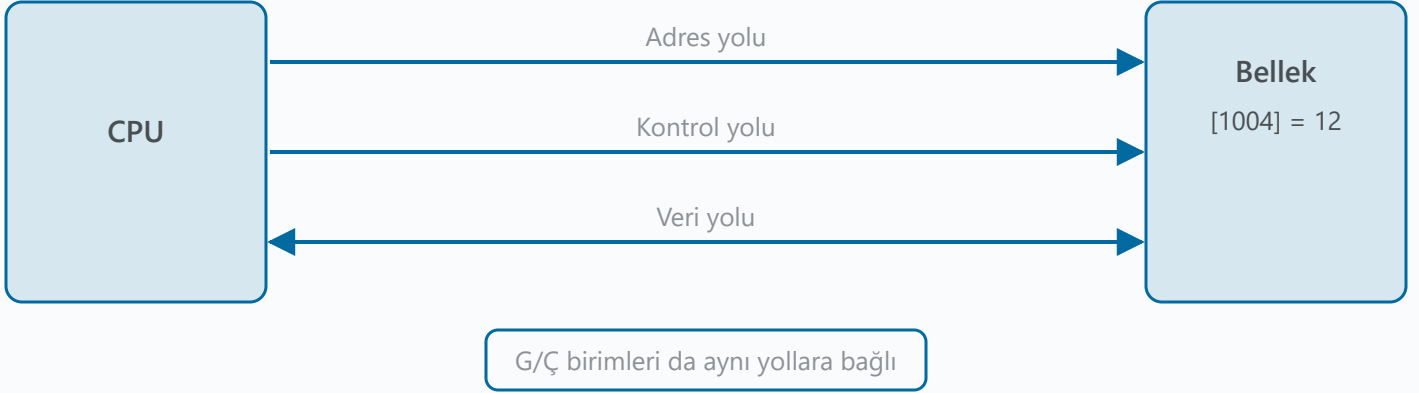
Yazmaçlar

İşlem sırasında kullanılan küçük ve hızlı depolama alanlarıdır.

Önbellek

Sık kullanılan veri ve komutları işlemciye yakın tutar.

Veri Yolları: Bellekten Okuma

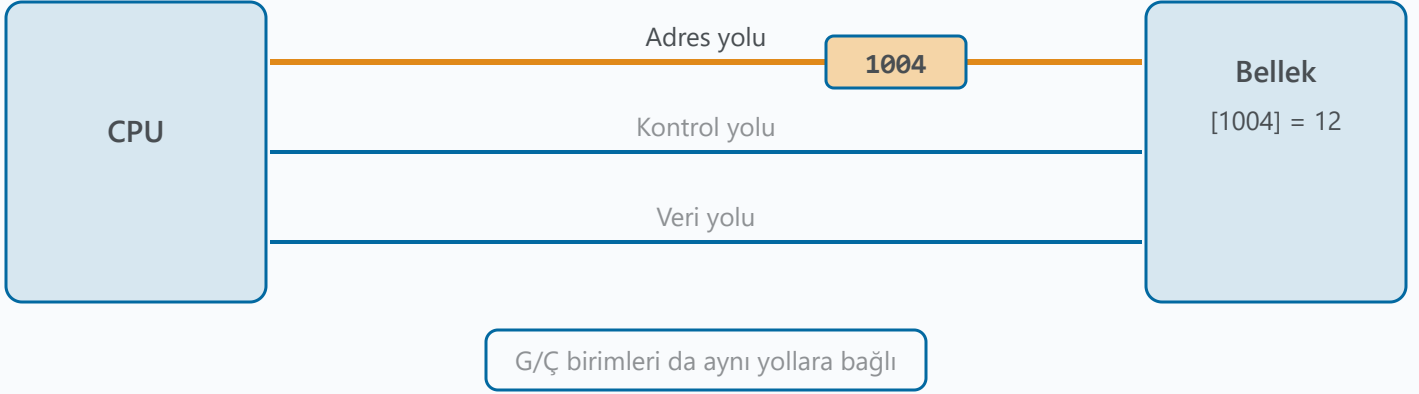


Üç yol CPU'yu bellek ve G/Ç birimlerine bağlar. Örnek: CPU, 1004 adresindeki değeri okuyacak.

Adım 1 / 4

Yol	Görev
Veri yolu	CPU, bellek ve aygıtlar arasında veriyi taşır.
Adres yolu	İşlem yapılacak bellek veya aygıt konumunu belirtir.
Kontrol yolu	Okuma, yazma ve zamanlama sinyallerini taşır.

Veri Yolları: Bellekten Okuma

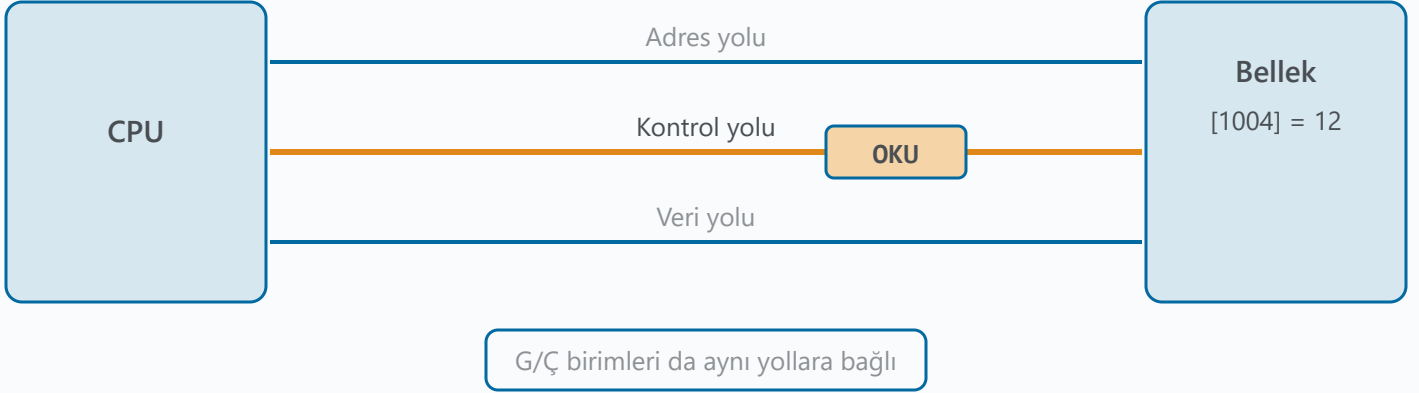


Adres yolu: CPU hangi hücreyi istediğini gönderir → **1004**. Tek yönlüdür (CPU'dan çıkar).

Adım 2 / 4

Yol	Görev
Veri yolu	CPU, bellek ve aygıtlar arasında veriyi taşır.
Adres yolu	İşlem yapılacak bellek veya aygıt konumunu belirtir.
Kontrol yolu	Okuma, yazma ve zamanlama sinyallerini taşır.

Veri Yolları: Bellekten Okuma

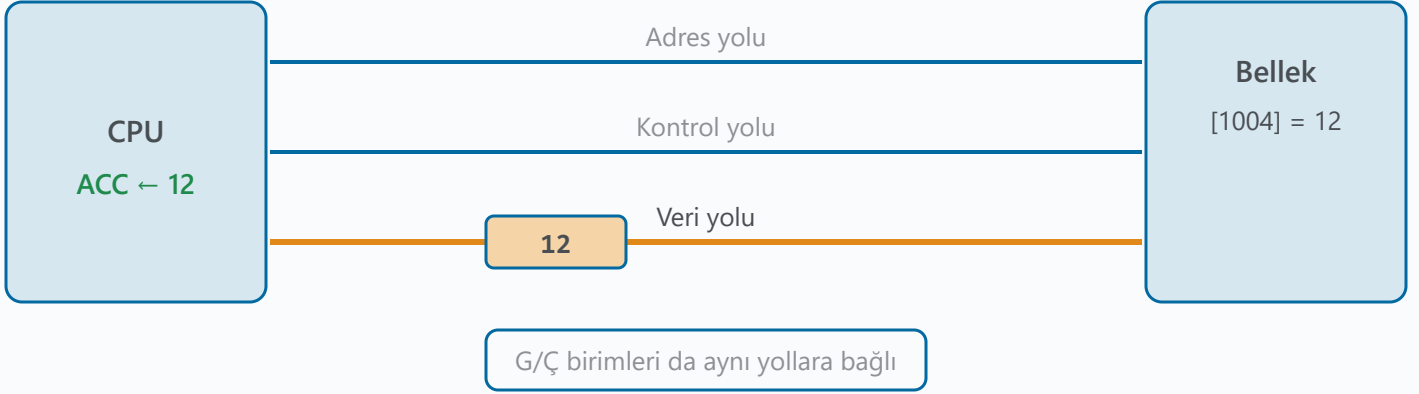


Kontrol yolu: CPU **OKU** sinyalini gönderir (yazma olsaydı YAZ). Zamanlama sinyalleri de bu yoldadır.

Adım 3 / 4

Yol	Görev
Veri yolu	CPU, bellek ve aygıtlar arasında veriyi taşır.
Adres yolu	İşlem yapılacak bellek veya aygıt konumunu belirtir.
Kontrol yolu	Okuma, yazma ve zamanlama sinyallerini taşır.

Veri Yolları: Bellekten Okuma



Veri yolu: bellek hücrenin içeriğini (**12**) CPU'ya gönderir. Veri yolu çift yönlüdür.

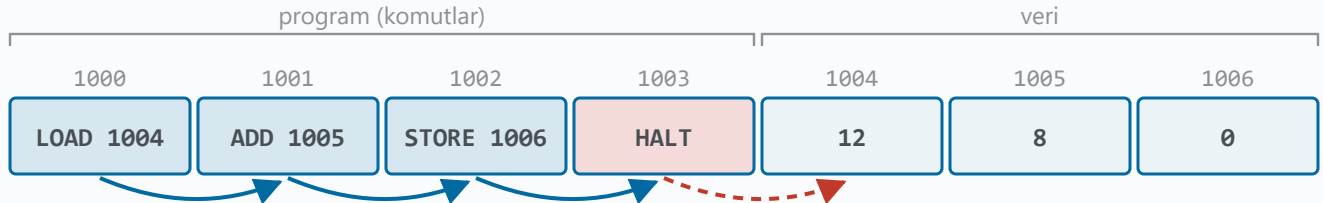
Adım 4 / 4

Yol	Görev
Veri yolu	CPU, bellek ve aygıtlar arasında veriyi taşır.
Adres yolu	İşlem yapılacak bellek veya aygıt konumunu belirtir.
Kontrol yolu	Okuma, yazma ve zamanlama sinyallerini taşır.

Bellek Adresleri

Adres	İçerik
1000	LOAD 1004
1001	ADD 1005
1002	STORE 1006
1003	HALT
1004	12
1005	8
1006	0

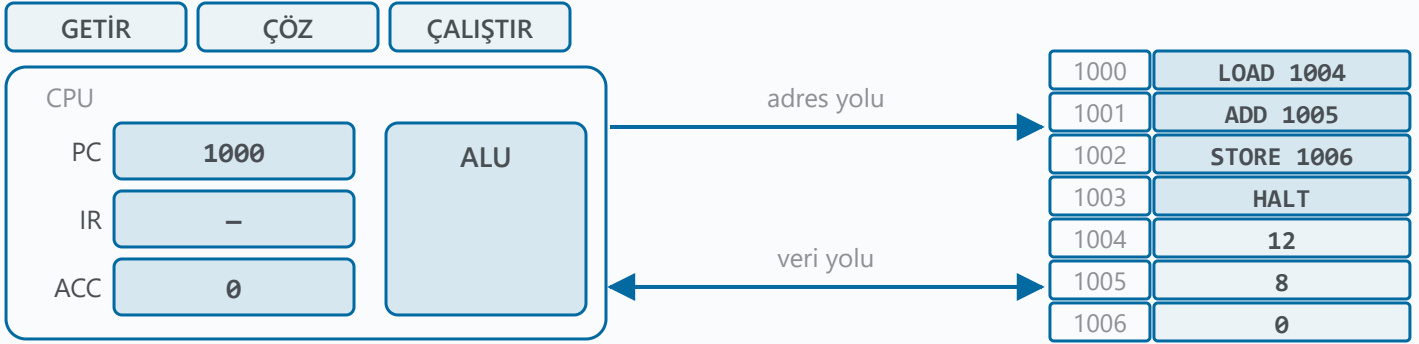
- Her bellek hücresinin bir adresi vardır.
- Komutlar adres kullanarak veriye ulaşır.
- Aynı bit dizisi komut veya veri olarak yorumlanabilir.
- **HALT** olmasaydı CPU 1004'teki 12'yi de komut sanıp çalıştırmaya çalışırdı.



PC sırayla ilerler: getir → çöz → çalıştır

HALT olmasaydı: 12 komut sanılırdı

Programı Adım Adım Çalıştırma



Başlangıç: **PC = 1000** (ilk komutun adresi), ACC = 0. CPU döngüye başlar.

Adım 1 / 12

```

▶ 1000: LOAD 1004 ; ACC ← bellek[1004]
2 1001: ADD 1005 ; ACC ← ACC + bellek[1005]
3 1002: STORE 1006 ; bellek[1006] ← ACC
4 1003: HALT ; dur
5 1004: 12 ; veri
6 1005: 8 ; veri
7 1006: 0 ; sonuç
  
```

DEĞİŞKENLER

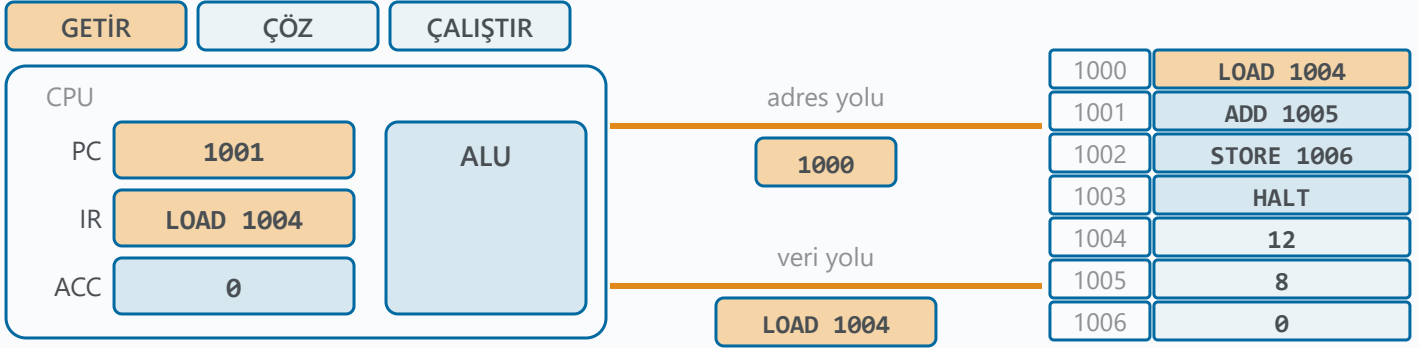
PC = **1000**

IR = **-**

ACC = **0**

bellek[1006] = **0**

Programı Adım Adım Çalıştırma



Getir: PC'deki 1000 adres yoluna çıkar, komut **LOAD 1004** veri yoluyla **IR**'ye gelir. PC bir artar → **1001**.

Adım 2 / 12

```

► 1000: LOAD 1004 ; ACC ← bellek[1004]
2 1001: ADD 1005 ; ACC ← ACC + bellek[1005]
3 1002: STORE 1006 ; bellek[1006] ← ACC
4 1003: HALT ; dur
5 1004: 12 ; veri
6 1005: 8 ; veri
7 1006: 0 ; sonuç

```

DEĞİŞKENLER

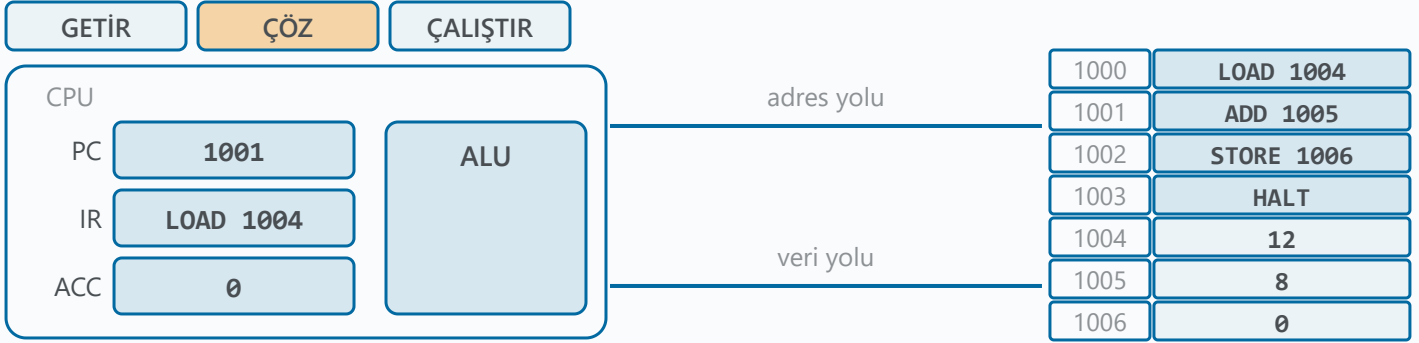
PC = 1001

IR = LOAD 1004

ACC = 0

bellek[1006] = 0

Programı Adım Adım Çalıştırma



Çöz: kontrol birimi IR'yi ayırır: opcode **LOAD**, adres **1004** → bellek[1004] okunup ACC'ye yüklenecek.

Adım 3 / 12

```

► 1000: LOAD 1004 ; ACC ← bellek[1004]
2 1001: ADD 1005 ; ACC ← ACC + bellek[1005]
3 1002: STORE 1006 ; bellek[1006] ← ACC
4 1003: HALT ; dur
5 1004: 12 ; veri
6 1005: 8 ; veri
7 1006: 0 ; sonuç
  
```

DEĞİŞKENLER

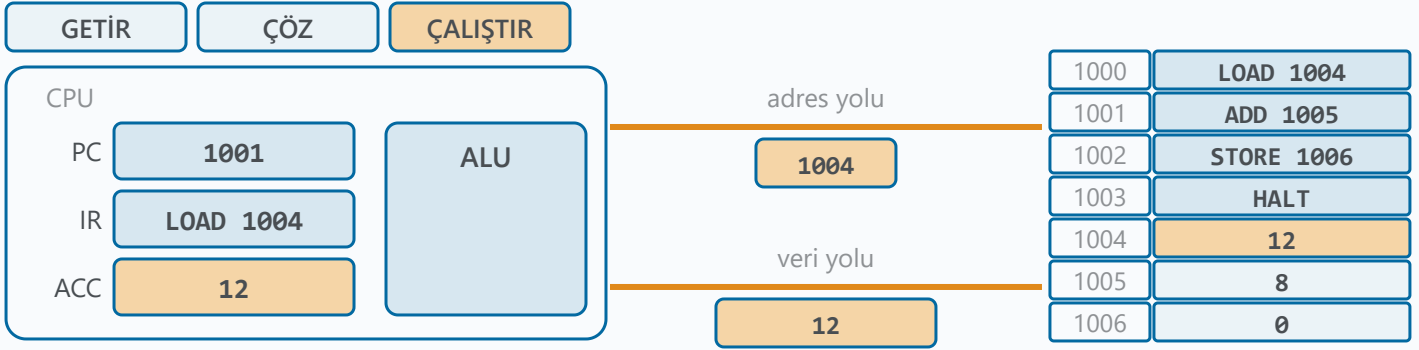
PC = 1001

IR = LOAD 1004

ACC = 0

bellek[1006] = 0

Programı Adım Adım Çalıştırma



Çalıştır: 1004 adresi okunur, **12** ACC'ye yüklenir.

Adım 4 / 12

```

▶ 1000: LOAD 1004 ; ACC ← bellek[1004]
2 1001: ADD 1005 ; ACC ← ACC + bellek[1005]
3 1002: STORE 1006 ; bellek[1006] ← ACC
4 1003: HALT ; dur
▶ 1004: 12 ; veri
6 1005: 8 ; veri
7 1006: 0 ; sonuç

```

DEĞİŞKENLER

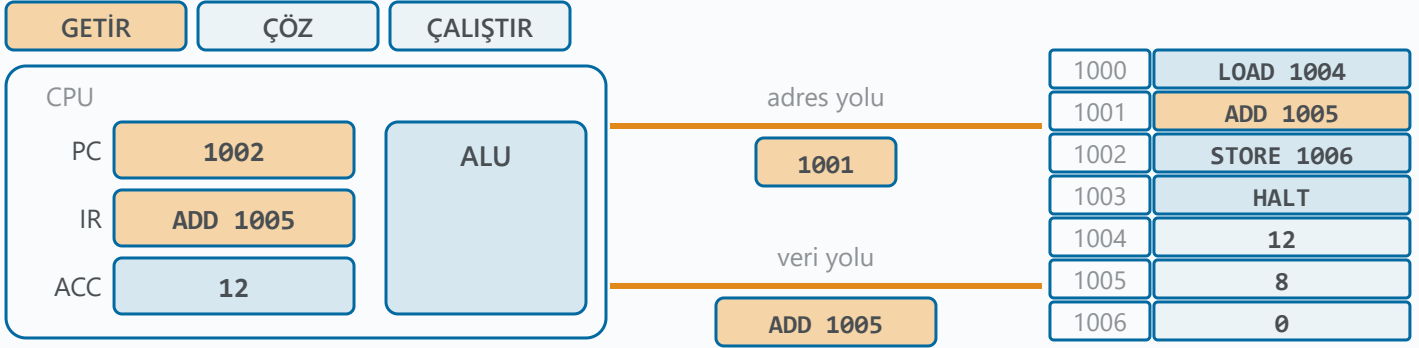
PC = 1001

IR = LOAD 1004

ACC = 12

bellek[1006] = 0

Programı Adım Adım Çalıştırma



Getir: PC'deki 1001 adres yoluna çıkar, komut **ADD 1005** veri yoluyla **IR**'ye gelir. PC bir artar → **1002**.

Adım 5 / 12

1	1000:	LOAD	1004	; ACC ← bellek[1004]
2	▶ 1001:	ADD	1005	; ACC ← ACC + bellek[1005]
3	1002:	STORE	1006	; bellek[1006] ← ACC
4	1003:	HALT		; dur
5	1004:	12		; veri
6	1005:	8		; veri
7	1006:	0		; sonuç

DEĞİŞKENLER

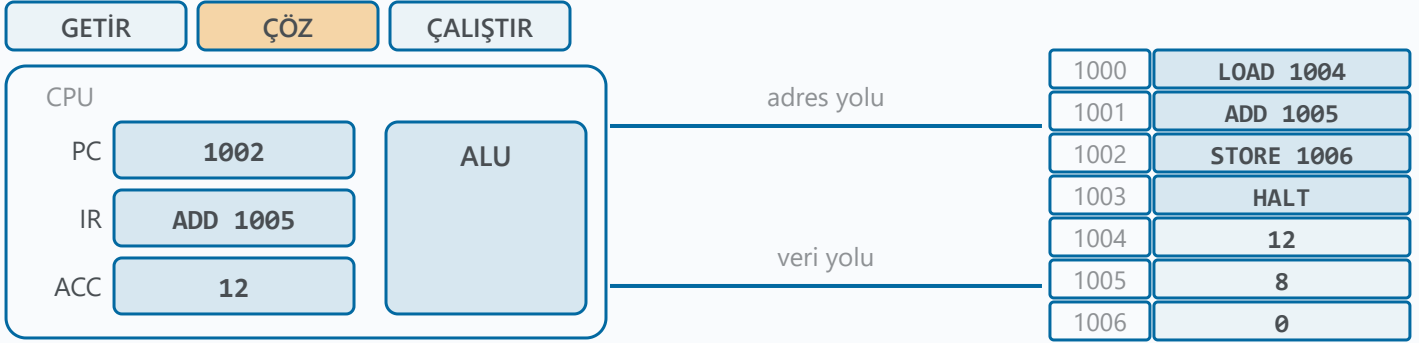
PC = 1002

IR = ADD 1005

ACC = 12

bellek[1006] = 0

Programı Adım Adım Çalıştırma



Çöz: kontrol birimi IR'yi ayırır: opcode **ADD**, adres **1005** → bellek[1005] ACC'ye eklenecek.

Adım 6 / 12

```

1 1000: LOAD 1004 ; ACC ← bellek[1004]
▶ 1001: ADD 1005 ; ACC ← ACC + bellek[1005]
3 1002: STORE 1006 ; bellek[1006] ← ACC
4 1003: HALT ; dur
5 1004: 12 ; veri
6 1005: 8 ; veri
7 1006: 0 ; sonuç

```

DEĞİŞKENLER

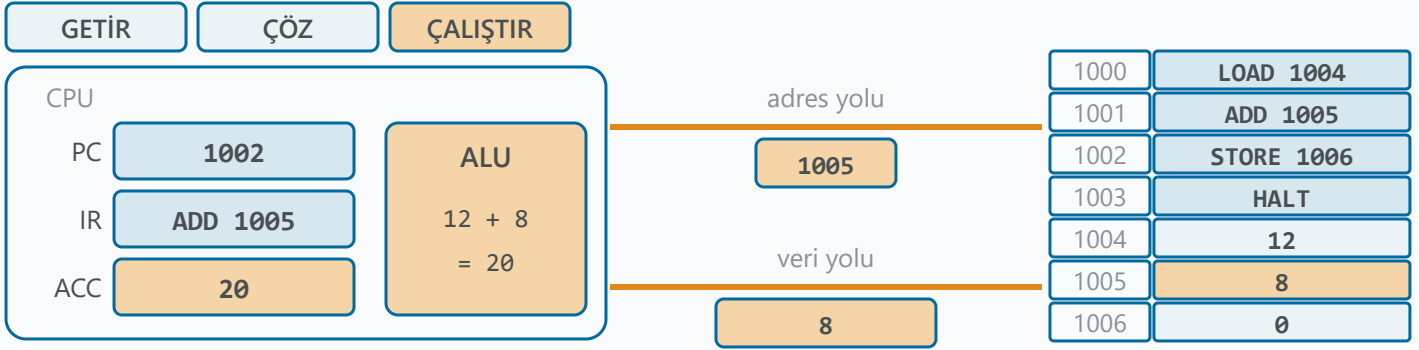
PC = 1002

IR = ADD 1005

ACC = 12

bellek[1006] = 0

Programı Adım Adım Çalıştırma



Çalıştır: 1005 adresindeki 8 okunur, **ALU** $12 + 8 = 20$ hesaplar, sonuç ACC'ye yazılır.

Adım 7 / 12

```

1 1000: LOAD 1004 ; ACC ← bellek[1004]
▶ 1001: ADD 1005 ; ACC ← ACC + bellek[1005]
3 1002: STORE 1006 ; bellek[1006] ← ACC
4 1003: HALT ; dur
5 1004: 12 ; veri
▶ 1005: 8 ; veri
7 1006: 0 ; sonuç
  
```

DEĞİŞKENLER

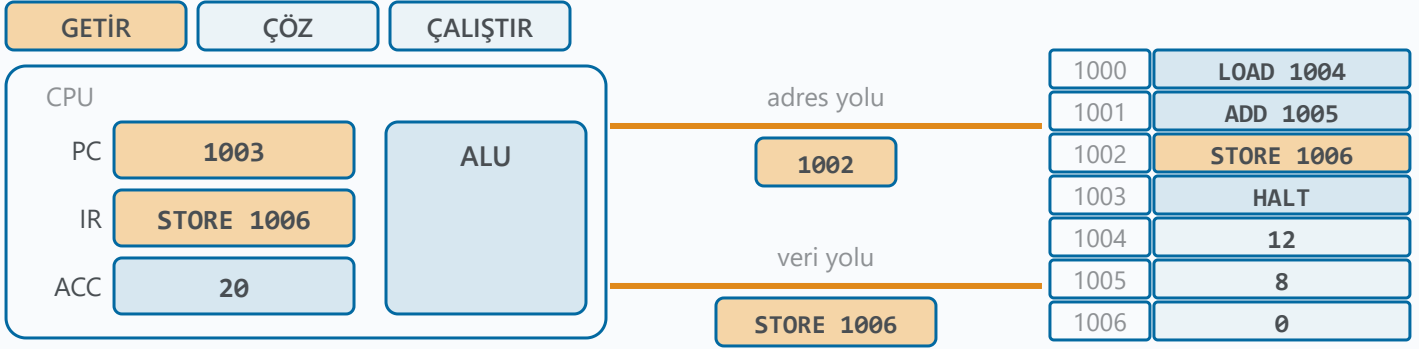
PC = 1002

IR = ADD 1005

ACC = 20

bellek[1006] = 0

Programı Adım Adım Çalıştırma



Getir: PC'deki 1002 adres yoluna çıkar, komut **STORE 1006** veri yoluyla **IR**'ye gelir. PC bir artar → **1003**.

Adım 8 / 12

```

1 1000: LOAD 1004 ; ACC ← bellek[1004]
2 1001: ADD 1005 ; ACC ← ACC + bellek[1005]
▶ 1002: STORE 1006 ; bellek[1006] ← ACC
4 1003: HALT ; dur
5 1004: 12 ; veri
6 1005: 8 ; veri
7 1006: 0 ; sonuç
  
```

DEĞİŞKENLER

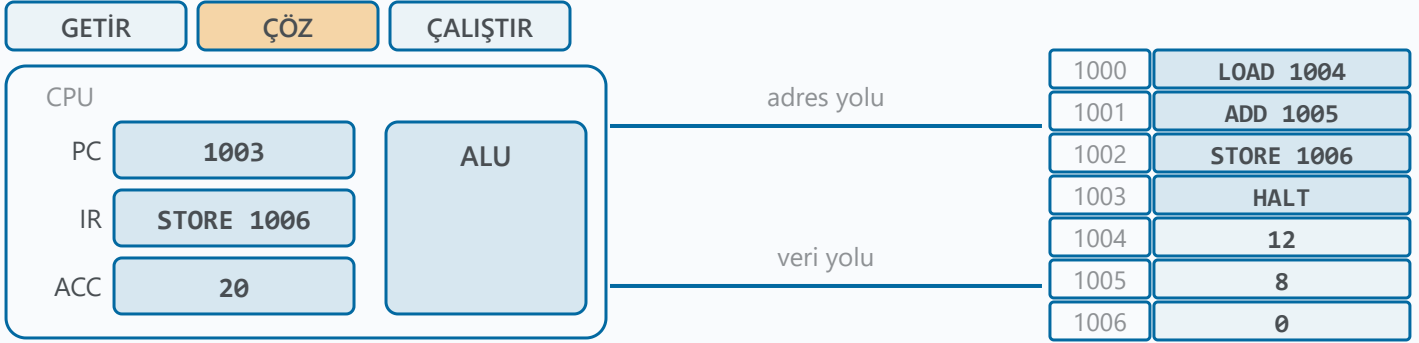
PC = 1003

IR = STORE 1006

ACC = 20

bellek[1006] = 0

Programı Adım Adım Çalıştırma



Çöz: kontrol birimi IR'yi ayırır: opcode **STORE**, adres **1006** → ACC bellek[1006]'e yazılacak.

Adım 9 / 12

```

1 1000: LOAD 1004 ; ACC ← bellek[1004]
2 1001: ADD 1005 ; ACC ← ACC + bellek[1005]
▶ 1002: STORE 1006 ; bellek[1006] ← ACC
4 1003: HALT ; dur
5 1004: 12 ; veri
6 1005: 8 ; veri
7 1006: 0 ; sonuç
  
```

DEĞİŞKENLER

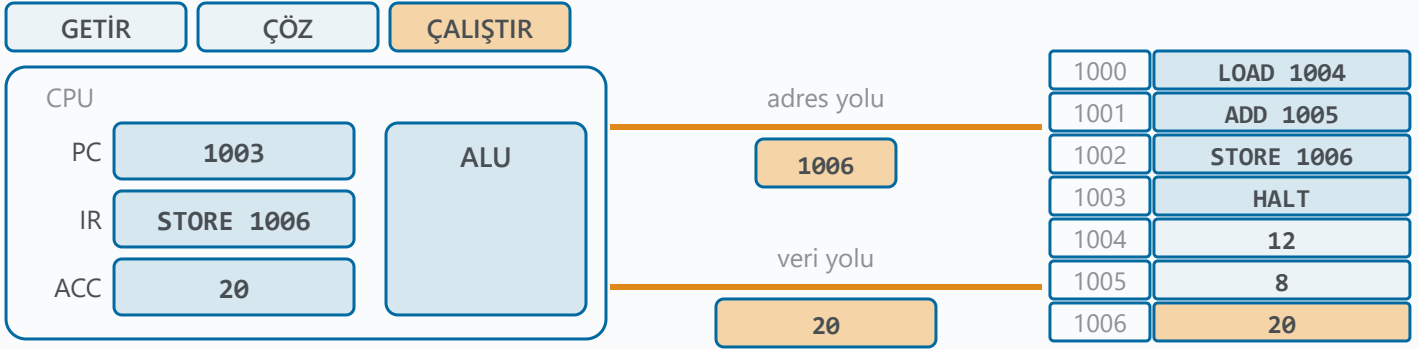
PC = 1003

IR = STORE 1006

ACC = 20

bellek[1006] = 0

Programı Adım Adım Çalıştırma



Çalıştır: ACC'deki **20** veri yoluyla 1006 adresine **yazılır**.

Adım 10 / 12

```

1 1000: LOAD 1004 ; ACC ← bellek[1004]
2 1001: ADD 1005 ; ACC ← ACC + bellek[1005]
▶ 1002: STORE 1006 ; bellek[1006] ← ACC
4 1003: HALT ; dur
5 1004: 12 ; veri
6 1005: 8 ; veri
▶ 1006: 0 ; sonuç

```

DEĞİŞKENLER

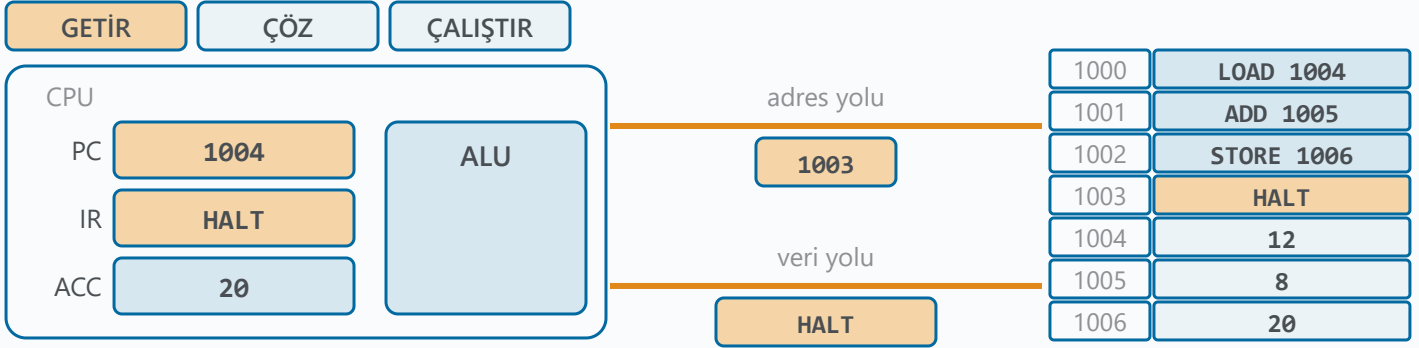
PC = **1003**

IR = **STORE 1006**

ACC = **20**

bellek[1006] = **20**

Programı Adım Adım Çalıştırma



Getir: PC'deki 1003 adres yoluna çıkar, komut **HALT** veri yoluyla **IR**'ye gelir. PC bir artar → **1004**.

Adım 11 / 12

```

1 1000: LOAD 1004 ; ACC ← bellek[1004]
2 1001: ADD 1005 ; ACC ← ACC + bellek[1005]
3 1002: STORE 1006 ; bellek[1006] ← ACC
▶ 1003: HALT ; dur
5 1004: 12 ; veri
6 1005: 8 ; veri
7 1006: 0 ; sonuç
  
```

DEĞİŞKENLER

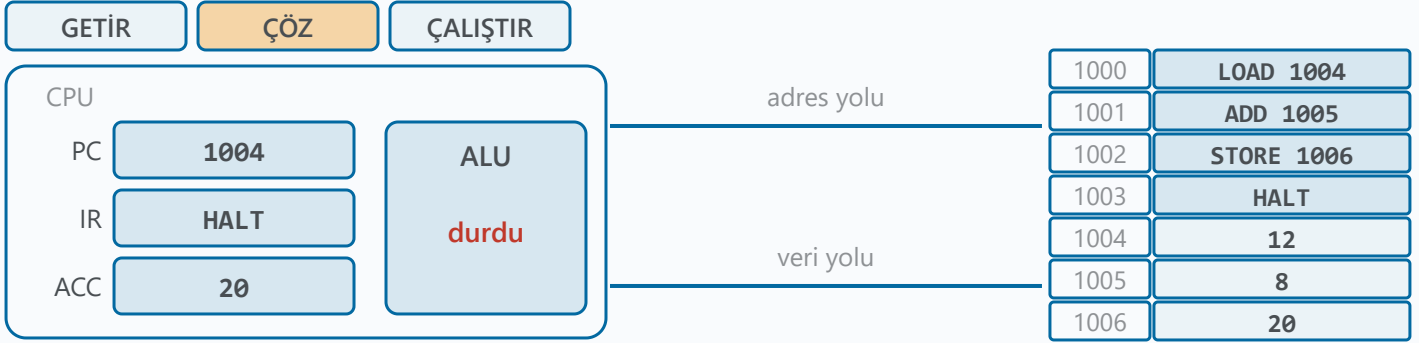
PC = 1004

IR = HALT

ACC = 20

bellek[1006] = 20

Programı Adım Adım Çalıştırma



Çöz: opcode HALT → CPU durur. Sonuç bellek[1006] = **20**. 4 komut, 4 getir-çöz-çalıştır turu.

Adım 12 / 12

```

1 1000: LOAD 1004 ; ACC ← bellek[1004]
2 1001: ADD 1005 ; ACC ← ACC + bellek[1005]
3 1002: STORE 1006 ; bellek[1006] ← ACC
▶ 1003: HALT ; dur
5 1004: 12 ; veri
6 1005: 8 ; veri
7 1006: 0 ; sonuç

```

DEĞİŞKENLER

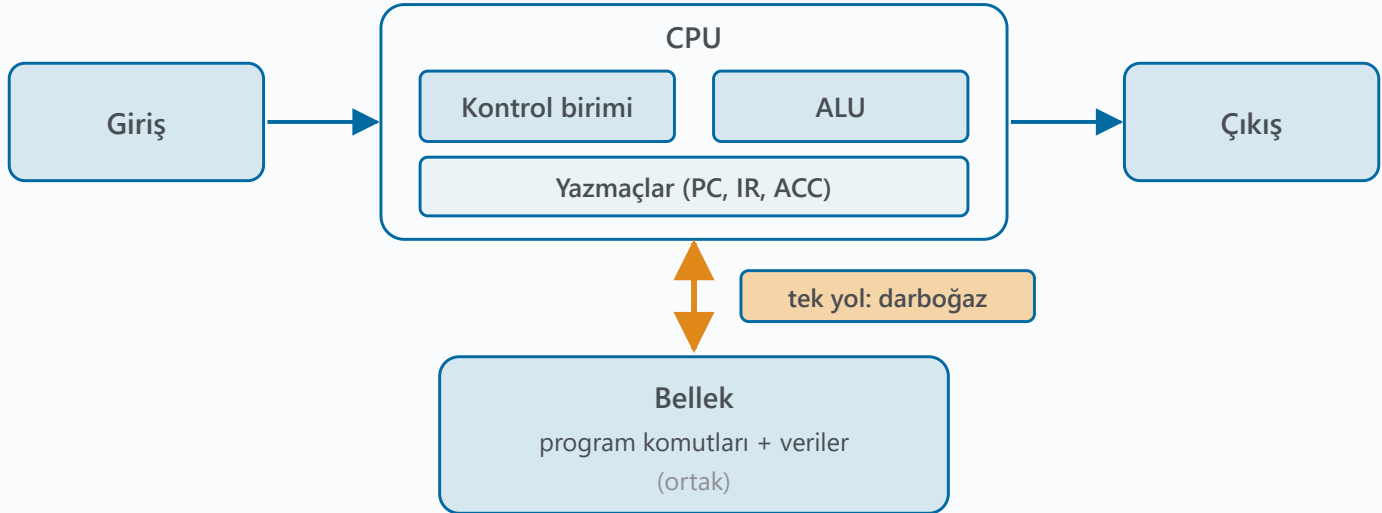
PC = 1004

IR = HALT

ACC = 20

bellek[1006] = 20

von Neumann Mimarisi



- Program komutları ve veriler ortak bellekte tutulur.
- CPU komutları sırayla işler; dallanma komutları bu sırayı değiştirebilir.
- Komut ve veri aynı yoldan geldiği için CPU–bellek aktarımı performansı sınırlar (**von Neumann darboğazı**).

Komutların Kodlanması

Makine komutu genellikle yapılacak işlemi ve kullanılacak değer veya adresleri belirtir.

ADD 1005 →



Örnek (varsayımsal) kodlar: HALT = 0000 · LOAD = 0001 · ADD = 0010 · STORE = 0011

Opcode (işlem kodu)

İşlemin türünü tanımlar.

ADD

İşlenen (operand)

İşlemde kullanılacak yazmaç, değer veya adres.

R1, R2

Komut biçimi

Bit alanlarının sayısını ve yerini mimari belirler.

Komut kümesi

İşlemcinin doğrudan anlayabildiği komutların tamamı.

Mimari Sınırlar

Sonlu bellek

Aynı anda saklanabilecek veri miktarı sınırlıdır.

Sonlu hız

Her komut zaman ve enerji tüketir.

Temsil hataları

Sınırlı bit sayısı taşma ve yuvarlama üretebilir.

Veri darboğazı

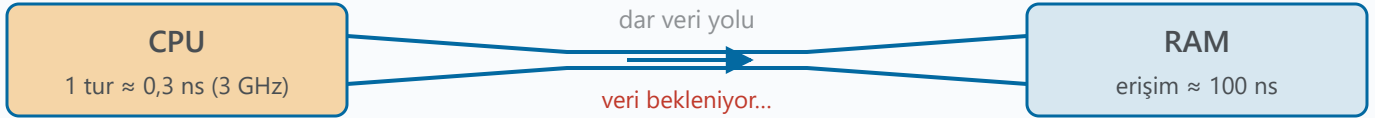
CPU, bellekteki veriyi beklerken boş kalabilir.

Paralellik sınırı

Her problem eşit ölçüde paralelleştirilemez.

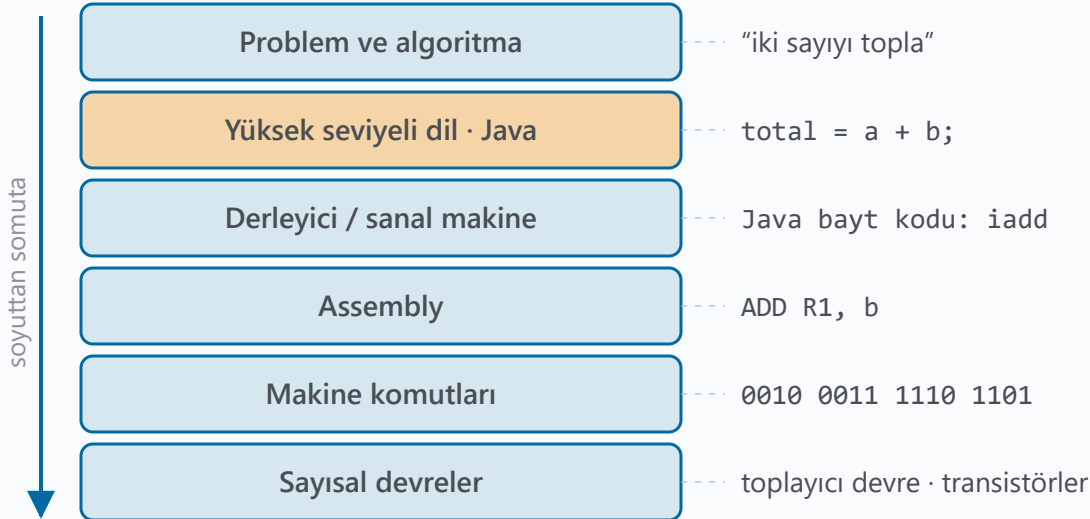
Güvenlik

Program ve veri aynı sistem kaynaklarını paylaştığı için yalıtım gerekir.



$100 \text{ ns} \div 0,3 \text{ ns} \approx 300$ tur: önbellek bu bekleyişi azaltmak için vardır

Programlama Soyutlama Düzeyleri



Her düzey bir alttakinin ayrıntısını gizler. Örnek satırlar gösterim amaçlıdır; gerçek kodlar mimariye göre değişir.

Yüksek Seviyeli Dil ve Assembly

Java

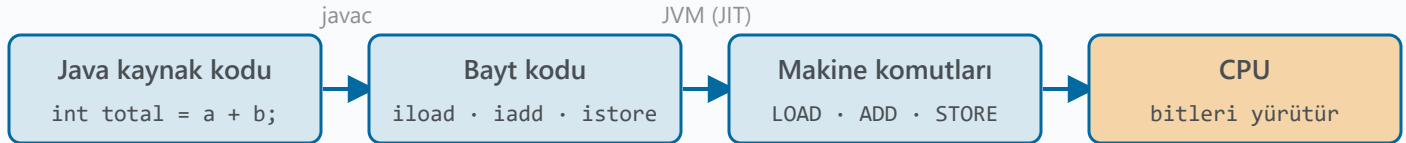
```
int total = a + b;
```

İnsan tarafından okunabilir, taşınabilir ve üretkenlik odaklıdır.

Assembly

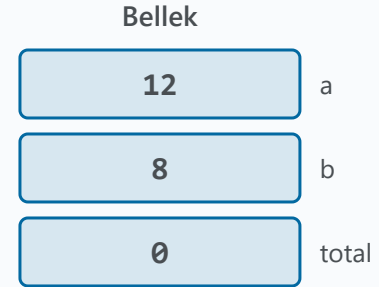
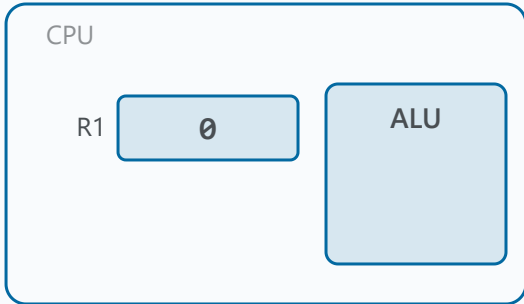
```
LOAD  R1, a  
ADD   R1, b  
STORE R1, total
```

Belirli işlemci mimarisine yakın ve ayrıntılıdır.



Makine komutları, sağdaki assembly'nin ikili karşılığıdır; işlemci mimarisine özgüdür.

Kod İzleme: `total = a + b`



Tek Java satırı işlemcide **üç** komuta dönüşür. Bellekte $a = 12$, $b = 8$ var.

Adım 1 / 4

► // Java: `int total = a + b;`

```
2 LOAD R1, a      ; R1 ← a
3 ADD  R1, b      ; R1 ← R1 + b
4 STORE R1, total ; total ← R1
```

DEĞİŞKENLER

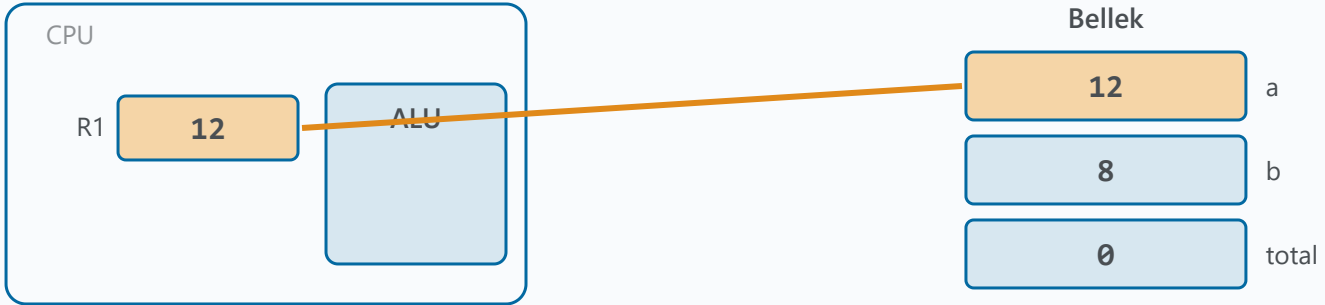
R1 = 0

a = 12

b = 8

total = 0

Kod İzleme: `total = a + b`



LOAD: a'nın değeri bellekten yazmaca taşınır → R1 = **12**. ALU yalnızca yazmaçlarla çalışır.

Adım 2 / 4

```
1 // Java: int total = a + b;
```

```
▶ LOAD R1, a ; R1 ← a
```

```
3 ADD R1, b ; R1 ← R1 + b
```

```
4 STORE R1, total ; total ← R1
```

DEĞİŞKENLER

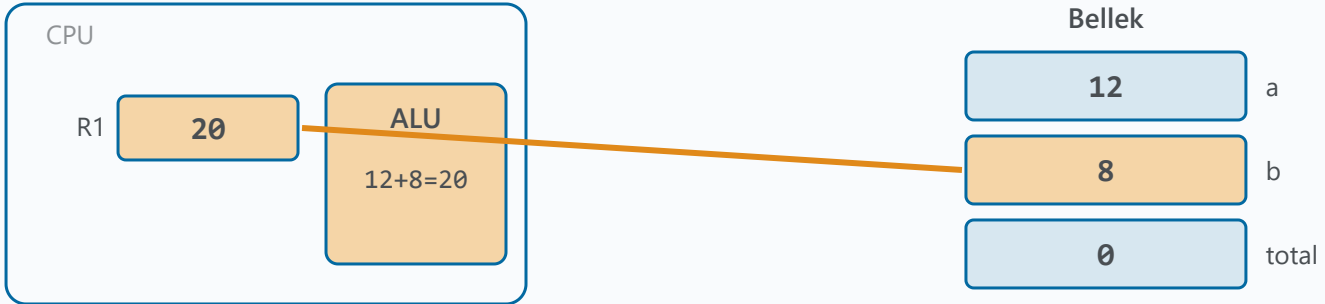
R1 = **12**

a = 12

b = 8

total = 0

Kod İzleme: `total = a + b`



ADD: b okunur, ALU $12 + 8 = 20$ hesaplar, sonuç R1'e yazılır.

Adım 3 / 4

```
1 // Java: int total = a + b;  
2 LOAD R1, a ; R1 ← a  
3 ADD R1, b ; R1 ← R1 + b  
4 STORE R1, total ; total ← R1
```

DEĞİŞKENLER

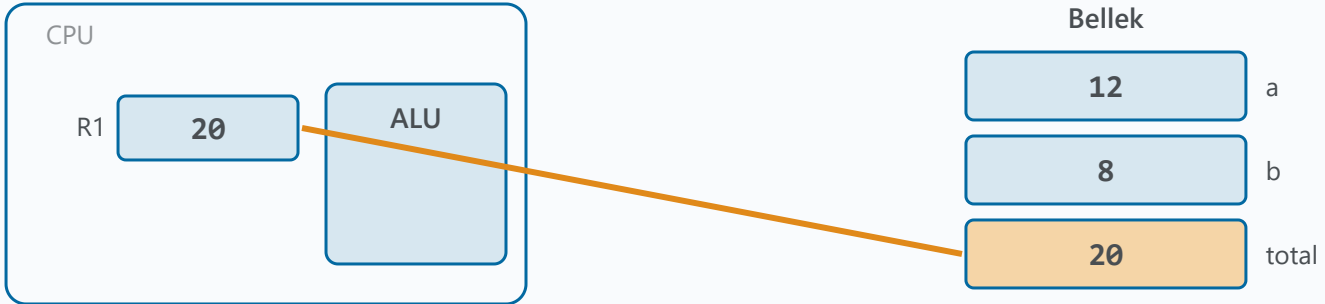
R1 = 20

a = 12

b = 8

total = 0

Kod İzleme: `total = a + b`



STORE: R1'deki 20 belleğe, total değişkeninin yerine yazılır. Java satırı tamamlandı: total = **20**.

Adım 4 / 4

```
1 // Java: int total = a + b;  
2 LOAD R1, a ; R1 ← a  
3 ADD R1, b ; R1 ← R1 + b  
4 STORE R1, total ; total ← R1
```

DEĞİŞKENLER

R1 = 20

a = 12

b = 8

total = 20

Bölüm Özeti

- Bilgisayar bütün verileri bit dizileriyle temsil eder.
- İkili gösterim tam sayılar için kesin, kayan nokta bazı ondalıklar için yaklaşık olabilir.
- ASCII ve Unicode karakterleri sayısal kodlarla eşleştirir.
- Saklanan program yaklaşımında komutlar ve veriler bellekte tutulur.
- CPU, komutları getir-çöz-çalıştır döngüsünde yürütür.
- Yüksek seviyeli diller donanım ayrıntılarını soyutlar.

Kontrol Soruları

1. 1101_2 sayısı onluk sistemde kaçtır?
2. 8 bit işaretli tam sayı aralığı nedir?
3. ASCII ile Unicode arasındaki temel fark nedir?
4. Program komutlarının bellekte saklanması hangi avantajı sağlar?
5. Veri, adres ve kontrol yollarının görevleri nelerdir?
6. Java kodu işlemcinin anlayacağı komutlara nasıl dönüşür?

Alıştırmalar

1. 8 bit ikiye tümleyen gösterimde 11111110 hangi sayıdır?
2. $3F_{16}$ sayısını ikili ve onluk sisteme çevirin.
3. "ç" (U+00E7) karakterini UTF-8 ile kodlayın; kaç bayt tutar?
4. 8 bit işaretli yazmaçta $250 + 10$ işleminin sonucu ne olur? Neden?
5. "Programı Adım Adım Çalıştırma" örneğinde $1004 = 7$ ve $1005 = 5$ olsaydı program bitince 1006'da ne olurdu? Kaç komut getirilirdi?
6. `int fark = a - b;` satırı için LOAD / SUB / STORE ile assembly karşılığını yazın.

► Cevapları göster

Kaynaklar

- Patterson ve Hennessy, Computer Organization and Design.
- Bryant ve O'Hallaron, Computer Systems: A Programmer's Perspective.
- Unicode Consortium, The Unicode Standard.
- IEEE 754, Floating-Point Arithmetic.

Son

Bilgisayarlar

Sonraki sunum: [Hesaplama ve Hesaplamalı Düşünme](#) · [Çalışma soruları](#) · Simülatörler: [Sayı Tabanı](#)
[Dönüştürücü](#), [Mantık Kapıları](#), [Basit CPU: Getir-Çöz-Çalıştır](#) · [Terim sözlüğü](#)